GUÍA DE ESTUDIO · TPI BASES DE DATOS

Cómo defender el modelo

Bodegas del Atuel

Teoría, decisiones y preguntas probables para el oral

Material de estudio · 22 slides

Jamardo, Camila (10842) · Sirotiuk, Juliana (10939)



Qué se evalúa (y cómo ordenar la respuesta)

LO QUE MIRAN

- Cómo identificaron las entidades y qué las identifica.
- Por qué algo es tabla propia y no una columna de otra.
- Relaciones y cardinalidades: 1:N, N:M, tabla intermedia.
- Cómo llegaron a 1FN–2FN–3FN y qué redundancia conservaron.
- Qué decisiones tomaron y qué alternativas descartaron.
- Qué regla vive en clave, constraint, trigger o procedimiento.

CUATRO PREGUNTAS QUE ORDENAN TODO

- **¿Qué entidades?** → 001, DER y MER.
- **¿Cómo se relacionan?** → cardinalidades y tablas puente.
- **¿Por qué está normalizado?** → dependencias y formas normales.
- **¿Dónde vive cada regla?** → 002 a 005.
- **¿Qué descartaron?** → la decisión también se defiende.

> **Un error que entienden y saben explicar vale más que un modelo perfecto que no pueden justificar.**

TEORÍA

Claves: la identidad de cada fila

Concepto	En Bodegas del Atuel
Superclave	Cualquier conjunto de columnas que identifica una fila.
Clave candidata	Superclave minimal; puede haber más de una por tabla.
PK	La candidata elegida. Subrogada (id, código, número) o natural (cuit, CP).
UK	Otra candidata. Ej.: documento activo del cliente; (id, tipo) en cliente.
FK	Referencia a una PK o UK; garantiza integridad referencial.
Clave compuesta	Renglones (documento, vino); parcela (finca, nro_parcela).
Integridad referencial	Toda FK apunta a una fila existente: la base rechaza huérfanos.

La UK (id, tipo) de cliente no es capricho: es el destino de la FK compuesta que hace disjunta la jerarquía.

TEORÍA

Dependencias funcionales y formas normales

LA TEORÍA

- **DF:** X → Y; el valor de X determina el de Y.
- **1FN:** atributos atómicos; lo multivaluado va a otra tabla.
- **2FN:** ningún atributo depende de parte de la clave.
- **3FN:** ningún atributo no clave depende de otro no clave.
- **BCNF:** toda DF parte de una clave candidata; acá no hizo falta.
- **Redundancia:** se permite si hay un mecanismo que la mantiene.

EN EL MODELO

- `composicion`: (vino, cepa) → porcentaje.
- `renglon_comprobante`: (comprobante, vino) → cantidad, precio.
- `parcela`: (finca, número) → cepa, campaña.
- `cliente`: id → tipo → descuento; por eso el descuento se calcula.
- **Deliberadas:** `subtotal_renglon` (generada), totales (trigger + CHECK), campaña (generada).

Guardar un derivado se justifica solo si podés decir quién lo mantiene consistente.

TEORÍA

Cardinalidades y tablas puente

1:N

Cliente → comprobante. Sala → empleado. Bodega → vino. Finca → parcela.
La FK vive del lado N: muchos comprobantes, un cliente.

N:M con atributos

Vino ↔ cepa: `composicion` guarda `porcentaje`.
El dato depende del par, no de una entidad sola.

N:M sin atributos

Vino ↔ maridaje: `vino_maridaje` solo tiene las dos claves.
Tabla puente pura: la PK es el par.

1:1 y 0:N

Vino ↔ `stock_deposito` (PK = vino). Renglones 0:N: la cabecera nace vacía y el trigger exige al menos uno.

Si el dato describe la relación y no a una entidad, su lugar es la tabla intermedia.

Esquema: entidades, claves y límites

QUÉ HAY

- 30 tablas: 29 operativas + `auditoria_precio`.
- Subrogadas: id, código, número. Naturales: cuit, código postal.
- PK compuestas: renglones, composición, stock, parcela.
- Jerarquía de cliente y ENUM de estado y tipo.
- `comprobante` es la cabecera; `renglon_comprobante`, el detalle.
- Historial: el renglón guarda `precio_unitario`.

TABLA PROPIA VS COLUMNA

- `maridaje` es tabla: N:M y se comparte entre vinos.
- `departamento` es columna: no tiene clave ni relaciones.
- `email` y `teléfono` simples: un solo canal principal.
- `stock_deposito` es tabla 1:1: cantidad y PK propia.
- `stock_sala` es tabla: cantidad por sala y vino.

Una tabla propia existe cuando tiene clave, atributos y relaciones propias.

Jerarquía de cliente y empleado

CLIENTE: SUPERTIPO + SUBTIPOS

- `cliente`: id PK, `tipo` ENUM, UK (id, tipo).
- Subtipos: PK `id_cliente`, CHECK de `tipo`, FK (id_cliente, tipo).
- Un cliente no puede estar en dos subtipos: la PK lo impide.
- Total y disjunta; la totalidad se controla con constraint trigger diferido.
- Descartado: tabla única con NULLs y CHECKs; 4 tablas sin supertipo.

EMPLEADO: UNA SOLA TABLA

- Descartada la jerarquía empleado_sala / encargado_regional.
- UK (cuit, tipo) como destino de la FK compuesta.
- `localidad` FK (cuit_encargado, tipo_encargado) + CHECK fijo.
- El CHECK fija `tipo_encargado` en `encargado_regional`.
- 1:N: un empleado trabaja en una sola sala (FK NOT NULL).

La jerarquía evita NULLs y permite una única FK desde comprobante.

Constraints: la regla en el esquema

CHECK

Dominio: precio > 0, graduación 5–20, cantidades y superficies > 0, descuento 0–1.
Se evalúa fila a fila, en cada INSERT o UPDATE.

UNIQUE

Vino (nombre, añada, volumen); documento activo del cliente; parcela por campaña.
También sirve como destino de una FK.

FK y acciones

CASCADE solo si la hija no existe sin el padre; RESTRICT para el historial (RN-14). SET NULL no se usó.

Lo que no puede un constraint

Mirar otras filas (stock), la fecha actual o una transición de estados: eso va en trigger.

Regla fila a fila → constraint. Regla que depende de otras filas o del momento → trigger.

Funciones: cálculo con nombre propio

QUÉ ENCAPSULAN

- `fn_descuento_cliente`: CASE por tipo + descuento especial.
- `fn_total_comprobante`: suma renglones y aplica descuento.
- `fn_stock_sala`: cantidad o 0 con COALESCE.
- `fn_transicion_valida`: confección → terminado → procesado.
- `fn_validar_composicion_vino`: suma exactamente 100.
- `fn_validar_subtipo_cliente`: un cliente, un subtipo.

VOLATILIDAD Y USO

- **IMMUTABLE:** solo depende de parámetros (transición).
- **STABLE:** lee datos, no los modifica (descuento, stock).
- **VOLATILE:** no se usa para cálculos repetibles.
- Se usan en triggers, procedimientos, vistas y tests.
- No reemplazan al constraint: evitan repetir lógica.

La función no reemplaza al constraint: encapsula un cálculo o validación reutilizable.

Procedimientos: una operación completa

p_procesar_ingreso

FOR UPDATE, estado terminado, exige renglones y hace upsert en `stock_deposito`. Evita el doble procesamiento.

p_procesar_transferencia

Descuenta depósito, acredita sala y valida stock y estado.
Evita dejar stock en negativo.

p_crear_cliente

Inserta supertipo y subtipo en una sola transacción. `p_cambiar_tipo_cliente` mueve de subtipo sin dejar huérfanos.

POR QUÉ EN LA BASE

- Son varias sentencias que deben ser atómicas: lock + validación + escritura.
- La API no puede garantizar concurrencia por sí sola; el procedimiento sí.

El procedimiento es una operación de negocio atómica; la API solo hace CALL.

Triggers: la regla que ve la transacción

VENTA, PASO A PASO

1. BEFORE INSERT: valida stock en la sala (RN-09).
2. AFTER INSERT: descuenta stock y recalcula totales (RN-08).
3. Si algo falla: excepción y rollback completo.
4. La API no envía totales: los pone la base.

OTRAS REGLAS EN TRIGGERS

- Transiciones de estado (RN-10 y RN-11).
- Renglones inmutables fuera de confección (RN-12).
- Fecha de confección fija (RN-11).
- Auditoría de precio (RN-13) y composición 100 (RN-05).
- Todo cliente cumple: API, psql o script.

El trigger no es magia: es una regla que necesita ver la fila y el estado de la transacción.

Vistas: definir una vez, leer siempre

vista_stock_total_por_vino

Depósito + salas con LEFT JOIN y COALESCE.
El vino sin stock aparece con cero.

vista_ventas_mensuales_por_sala

Agregación mensual para reportes.
Una sola definición del indicador.

vista_documentos_pendientes

Ingresos y transferencias sin procesar.
Es la bandeja de trabajo.

QUÉ DEFENDER

- Una vista no guarda datos: guarda la definición; siempre muestra el estado actual.
- El LEFT JOIN mantiene los vinos sin stock; no se materializa porque los datos son operativos.

La vista no duplica datos: duplica la definición una sola vez.

Roles: privilegio mínimo

ROL_API

- DML sobre las 29 tablas operativas.
- SELECT e INSERT en `auditoria_precio`.
- EXECUTE de funciones y procedimientos.
- Nunca se conecta como owner.
- La API reporta `current_user = rol_api`.

ROL_REPORTE

- Solo SELECT sobre las 3 vistas.
- Sin acceso directo a las tablas base.
- Claves por entorno: APP_DB_PASSWORD.
- Un error o una inyección dañan menos.

El privilegio mínimo limita el daño de un error o de una inyección.

MIGRACIÓN 008

Índices: medir antes de crear

Consulta	Antes	Después	Mejora
S3	9,753 ms	0,977 ms	10,0×
S8	15,490 ms	9,174 ms	1,69×
S9	77 buf	3 buf	25,7×

CÓMO SE MIDE

- `EXPLAIN (ANALYZE, BUFFERS)` antes y después.
- Se comparan planes y buffers, no sensaciones.
- Se documenta también lo que no mejoró.

QUÉ DEFENDER

- B-tree: igualdad, rango y orden.
- S3 (sala, fecha) + INCLUDE: Index Only Scan.
- S8 (fecha, sala) + INCLUDE (total): cobertura.
- Parcial: transferencias pendientes. Expresión: lower(nombre).
- No todos ayudan: S1, S11 y S12 siguen con Seq Scan.

El índice se justifica con el plan y los buffers, no con la intuición.

RN-01 a RN-08: qué mecanismo y por qué

Regla	Qué exige	Mecanismo
RN-01	Precio del vino mayor que 0	CHECK
RN-02	Sala → localidad → encargado regional	FK + CHECK del tipo
RN-03	Cada empleado trabaja en una sola sala	FK NOT NULL
RN-04	Parcela: una cepa y una cosecha por campaña	FK + UNIQUE
RN-05	La composición del vino suma 100	Función + trigger diferido
RN-06	Descuento según tipo de cliente	Función + CHECK
RN-07	Vencimiento del permiso posterior a la emisión	CHECK
RN-08	Total del comprobante calculado	Trigger + función + CHECK

> **La regla se valida donde no se puede saltear: por eso la base y no la API.**

RN-09 a RN-15: qué mecanismo y por qué

Regla	Qué exige	Mecanismo
RN-09	No vender más que el stock de la sala	CHECK + trigger BEFORE
RN-10	Solo un ingreso terminado impacta el depósito	Trigger + procedimiento
RN-11	Transferencia, fecha de confección y stock	Trigger + procedimiento
RN-12	Renglones inmutables fuera de confección	Trigger
RN-13	Todo cambio de precio queda auditado	Trigger AFTER
RN-14	No borrar entidades con movimientos	FK ON DELETE RESTRICT
RN-15	Rangos y valores positivos	CHECK

Si la regla depende de otras filas o del momento, el constraint no alcanza: va trigger.

Qué se guarda y qué se calcula

Atributo	¿Se guarda?	Consistencia
Descuento del cliente	No	fn_descuento_cliente
descuento_especial	Sí, solo exportador	CHECK 0.30–0.35
subtotal_renglon	Sí	Columna generada
subtotal / descuento / total	Sí	Trigger + CHECK
Campaña del lote	Sí	Columna generada
Antigüedad	No	fn_antiguedad_anios
stock_total	No	Vista

Guardar un derivado se paga con mantenimiento; se acepta por historial o performance.

Decisiones y alternativas descartadas

Decisión	Alternativa descartada	Por qué
Cliente: supertipo + subtipos	Tabla única o 4 tablas sueltas	Evita NULLs y permite una FK única
Empleado: tabla única + ENUM	Jerarquía de roles	No hay atributos distintos
Maridaje: N:M	Columna multivaluada	Un maridaje se comparte entre vinos
Cliente–sala: vía comprobante	N:M directa	Evita pares sin comprobante
Ingreso–bodega: FK directa	Deducirla del vino	La cabecera la necesita y RN-14
Stock: 1:1 con vino	Entidad depósito	Hay un solo depósito
Precio: guardado en el renglón	Calcularlo al leer	Conserva el histórico

La alternativa descartada también se estudia: muestra que la decisión fue una elección.

Capa fina: la regla no vive acá

QUÉ HACE

- Front Nginx → API FastAPI → PostgreSQL.
- SQL parametrizado en `repositories`; sin ORM.
- Traduce SQLSTATE a HTTP (409, 400, etc.).
- Abre una transacción por operación.

QUÉ NO HACE

- No calcula totales ni descuentos.
- No valida stock ni transiciones.
- No decide reglas: las delega en la base.
- `/salud` confirma `current_user = rol_api`.

Si la pregunta es de API: capa fina, sin reglas de negocio.

ORAL

Preguntas probables I

¿Por qué cliente tiene UK (id, tipo)?

Es el destino de la FK compuesta de cada subtipo; garantiza que el tipo coincida y que un cliente esté en una sola subtabla.

¿Por qué no una sola tabla cliente?

La mayoría de las filas tendría NULLs; los subtipos tienen atributos propios y el descuento especial solo aplica a exportador.

¿Cómo aseguran que todo cliente tenga subtipo?

Con un constraint trigger diferido que, al cerrar la transacción, verifica que exista la fila del subtipo.

¿Dónde evitaron una dependencia parcial?

En composición y en los renglones. El descuento se movió del renglón al comprobante para respetar 2FN.

¿Por qué guardan totales derivados?

Por performance e histórico; los mantiene un trigger y un CHECK entre columnas de la misma fila.

¿Por qué el precio está en el renglón?

Porque el precio del vino cambia; el comprobante debe conservar el precio con el que se vendió.

ORAL

Preguntas probables II

¿Cuándo hace falta una tabla intermedia?

Cuando la relación es N:M o cuando tiene atributos propios, como el porcentaje de composición o la cantidad de stock.

¿Por qué stock_sala tiene PK compuesta?

Porque la cantidad depende del par sala + vino; ninguna de las dos columnas identifica sola la fila.

¿Constraint o trigger?

Constraint si la regla se decide fila a fila; trigger si necesita otras filas, la fecha actual o una transición.

¿Por qué el stock se controla en la base?

Porque cualquier cliente puede conectarse y la validación debe ser atómica con la venta: si falla, rollback.

¿Por qué ese orden de columnas en el índice?

Primero la igualdad o el rango según la consulta; INCLUDE agrega columnas para resolver con Index Only Scan.

¿Qué alternativa descartaron?

Cliente–sala N:M, depósito como entidad y jerarquía de empleado. Cada una tenía una razón de negocio.

CIERRE

Checklist para el oral

CADA UNO PUEDE EXPLICAR

- La jerarquía de cliente y su UK (id, tipo).
- Por qué cada regla está donde está.
- El flujo completo de una venta.
- Qué se guarda y qué se calcula.
- El plan de S3 y por qué baja.

REPARTO SUGERIDO

- Camila: modelo, normalización y 001–003.
- Juliana: reglas, 004–008 y demo.
- Las dos: demo y cierre.
- Ensayo: 3 corridas de 12 minutos.

No memoricen: sepan ubicar el archivo, la regla y la decisión.